# CHAPTER 8
# ENCODERS, DECODERS, MULTIPLEXERS, AND DEMULTIPLEXERS

## 8.1 Introduction

Discrete quantities of information are represented in digital systems with binary codes. Sometimes binary codes are used for error detection during digital data transmission. A binary code of $n$ bits is capable of representing up to $2^n$ distinct elements of the coded information. Discrete inputs to digital systems are encoded to binary-coded form by encoder circuits. In a reverse process, binary-coded data are decoded to discrete information by decoder circuits. Sometimes code conversion between one form to another is needed and is done by special types of encoder called code converter.

Multiplexing means transmitting a large number of information units over a smaller number of channels or lines. Demultiplexing is the reverse process of multiplexing. In demultiplexing process, information received on a smaller number of channels or lines is distributed over a large number of units. Multiplexer and demultiplexer circuits play an important role in digital systems where selection of path between multiple sources and single destination or between a single source and multiple destinations is required.

In this chapter design and application of various encoder, decoder, multiplexer, and demultiplexer circuits are studied.

## 8.2 Encoders

### 8.2.1 Design of Encoders

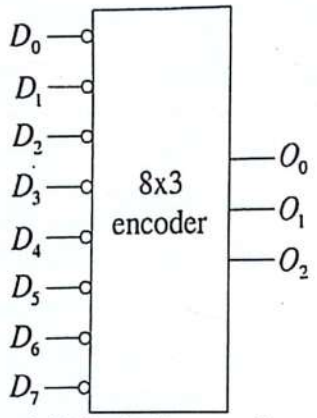
An *encoder* is a combinational circuit that has $2^n$ (or less) input lines and $n$ output lines. Only one of the input lines is activated at a given time. The output lines generate the binary code corresponding to the activated input. The input lines can be activated in two ways – by setting the activated input to 0 and the other inputs to 1 (active-LOW input) or by setting the activated input to 1 and the other inputs to 0 (active-HIGH input). The block diagram and truth tables of an *octal-to-binary encoder* (also called *8-to-3-line encoder*) for both active-LOW and active-HIGH inputs are shown in Figure 8.1. From the truth tables, the Boolean function for the outputs can be written as below.

For active-LOW input

$$O_0 = D_1' + D_3' + D_5' + D_7' = (D_1 D_3 D_5 D_7)'$$
$$O_1 = D_2' + D_3' + D_6' + D_7' = (D_2 D_3 D_6 D_7)'$$
$$O_2 = D_4' + D_5' + D_6' + D_7' = (D_4 D_5 D_6 D_7)'$$

For active-HIGH input

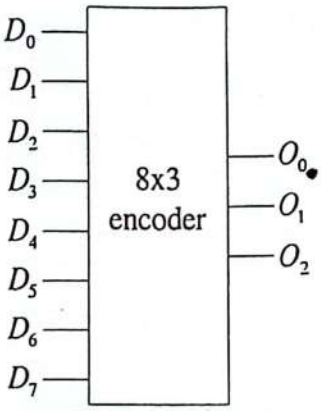
$$O_0 = D_1 + D_3 + D_5 + D_7$$
$$O_1 = D_2 + D_3 + D_6 + D_7$$
$$O_2 = D_4 + D_5 + D_6 + D_7$$




(a) Block diagram for active-LOW input

| Inputs | | | | | | | | Outputs | | |
|---|---|---|---|---|---|---|---|---|---|---|
| $D_7$ | $D_6$ | $D_5$ | $D_4$ | $D_3$ | $D_2$ | $D_1$ | $D_0$ | $O_2$ | $O_1$ | $O_0$ |
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 |
| 1 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 | 0 | 1 | 1 | 0 | 1 | 0 |
| 1 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 0 | 0 |
| 1 | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 1 |
| 1 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 |
| 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |

(b) Truth table for active-LOW input




(c) Block diagram for active-HIGH input

| Inputs | | | | | | | | Outputs | | |
|---|---|---|---|---|---|---|---|---|---|---|
| $D_7$ | $D_6$ | $D_5$ | $D_4$ | $D_3$ | $D_2$ | $D_1$ | $D_0$ | $O_2$ | $O_1$ | $O_0$ |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 |
| 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 1 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 0 | 0 |
| 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 1 |
| 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 |

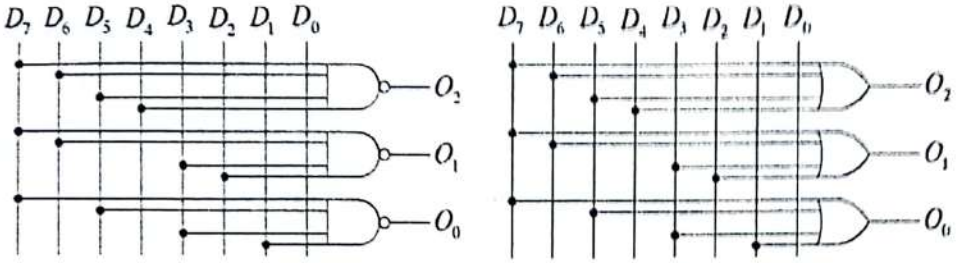
(d) Truth table for active-HIGH input

**Figure 8.1** Block diagram and truth table of octal-to-binary encoder (8-to-3-line encoder).

The logic implementation of the octal-to-binary encoder for both active-LOW and active-HIGH inputs is shown in Figure 8.2. Note that $D_0$ input is not connected to gates.

The encoder of Figure 8.2 assumes that only one input line can be activated at any time; otherwise the circuit has no meaning. Note that the circuit has eight inputs and could have $2^8 = 256$ possible input combinations. Only eight of these combinations have any meaning. The other input combinations are don't care

conditions. Encoders of this type are not available in IC package, since they can be easily constructed with gates. The type of encoders available in IC form is called a *priority encoder*. These encoders establish an input priority to ensure that only the highest-priority input line is encoded. Thus, in truth tables of Figure 8.1, if priority is given to an input with higher subscript number over one with a lower subscript number, then if both $D_2$ and $D_5$ are activated simultaneously, the output will be 101 because $D_5$ has a higher priority over $D_2$. Of course, the truth table of a priority encoder is different from the one of Figure 8.1.




(a) Active-LOW input (b) Active-HIGH input

**Figure 8.2** Logic implementation of octal-to-binary encoder (8-to-3-line encoder).

## 8.2.2 Available MSI Encoder ICs

Available MSI priority encoder ICs are given in Table 8.1.

**Table 8.1** Available MSI encoder ICs.

| IC No. | Description |
|---|---|
| 74147 | Decimal-to-BCD priority encoder with active-LOW input and inverted outputs. |
| 74148 | Octal-to-binary priority encoder with active-LOW input and inverted outputs. |

IC 74147 is introduced here. The pin assignment of IC 74147 is shown in Figure 8.3. IC 74147 9-input priority encoder accepts data from nine active-LOW inputs ($D_1'$ - $D_9'$) and provides a binary representation on the four active-LOW outputs ($O_0'$ - $O_3'$). A priority is assigned to each input so that when two or more inputs are simultaneously active, the input with the highest priority is represented on the output, with input line $D_9'$ having the highest priority. The IC provides the 10-to-4-line priority encoding function by use of the implied decimal "zero". The "zero" is encoded when all nine data inputs are HIGH, forcing all four outputs HIGH.

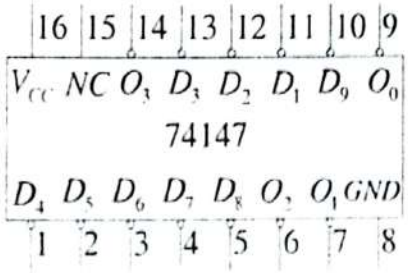



**Figure 8.3** IC type 74147 Decimal-to-BCD priority encoder

## 8.3 Decoders

### 8.3.1 Design of Decoders

A *decoder* is a combinational circuit that accepts $n$ inputs that represent a binary code and activate only the output that corresponds to the input code. The number of output lines may be a maximum of $2^n$. If the $n$-bit decoded information has unused or don't care combinations, the decoder output will have less than $2^n$ outputs. The output lines can be activated in two ways – by setting the activated output to 0 and the other outputs to 1 (active-LOW output) or by setting the activated output to 1 and the other outputs to 0 (active-HIGH output). If the number of output lines is less than $2^n$, the decoder is often designed such that all the outputs remain inactivated when an invalid input combination (unused code) is applied.

#### Binary-to-octal decoder

The block diagrams and truth tables of a *binary-to-octal decoder* (also called *3-to-8-line decoder* or *1-of-8 decoder*) for both active-LOW and active-HIGH outputs are shown in Figure 8.4. From the truth tables, the Boolean functions for the outputs can be written as below.

For active-LOW output

$$O_0 = A_2 + A_1 + A_0 = (A_2' A_1' A_0')'$$
$$O_1 = A_2 + A_1 + A_0' = (A_2' A_1' A_0)'$$
$$O_2 = A_2 + A_1' + A_0 = (A_2' A_1 A_0')'$$
$$O_3 = A_2 + A_1' + A_0' = (A_2' A_1 A_0)'$$
$$O_4 = A_2' + A_1 + A_0 = (A_2 A_1' A_0')'$$
$$O_5 = A_2' + A_1 + A_0' = (A_2 A_1' A_0)'$$
$$O_6 = A_2' + A_1' + A_0 = (A_2 A_1 A_0')'$$
$$O_7 = A_2' + A_1' + A_0' = (A_2 A_1 A_0)'$$

For active-HIGH output

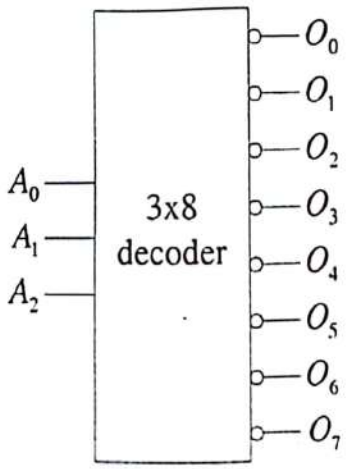
$$O_0 = A_2' A_1' A_0'$$
$$O_1 = A_2' A_1' A_0$$
$$O_2 = A_2' A_1 A_0'$$
$$O_3 = A_2' A_1 A_0$$
$$O_4 = A_2 A_1' A_0'$$

$$O_5 = A_2 A_1' A_0$$

$$O_6 = A_2 A_1 A_0'$$

$$O_7 = A_2 A_1 A_0$$

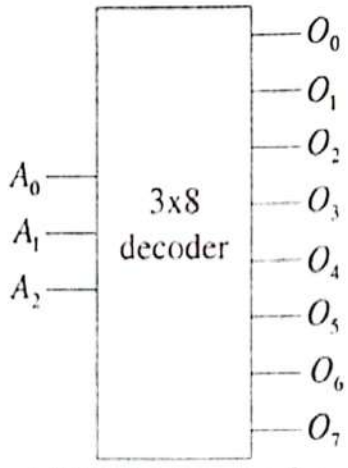
The logic implementation of the binary-to-octal decoder for both active-LOW and active-HIGH outputs is shown in Figure 8.5. Note that the outputs of the decoder with active-HIGH output are simply inverted to obtain the decoder with active-LOW output.




(a) Block diagram for active-LOW output

| Inputs | | | Outputs | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| $A_2$ | $A_1$ | $A_0$ | $O_7$ | $O_6$ | $O_5$ | $O_4$ | $O_3$ | $O_2$ | $O_1$ | $O_0$ |
| 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 1 |
| 0 | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 1 |
| 0 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 1 |
| 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 1 |
| 1 | 1 | 0 | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 1 |
| 1 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |

(b) Truth table for active-LOW output




(c) Block diagram for active-HIGH output

| Inputs | | | Outputs | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| $A_2$ | $A_1$ | $A_0$ | $O_7$ | $O_6$ | $O_5$ | $O_4$ | $O_3$ | $O_2$ | $O_1$ | $O_0$ |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |
| 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| 1 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| 1 | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

(d) Truth table for active-HIGH output

**Figure 8.4** Block diagram and truth table of binary-to-octal decoder (3-to-8-line decoder or 1-of-8 decoder).

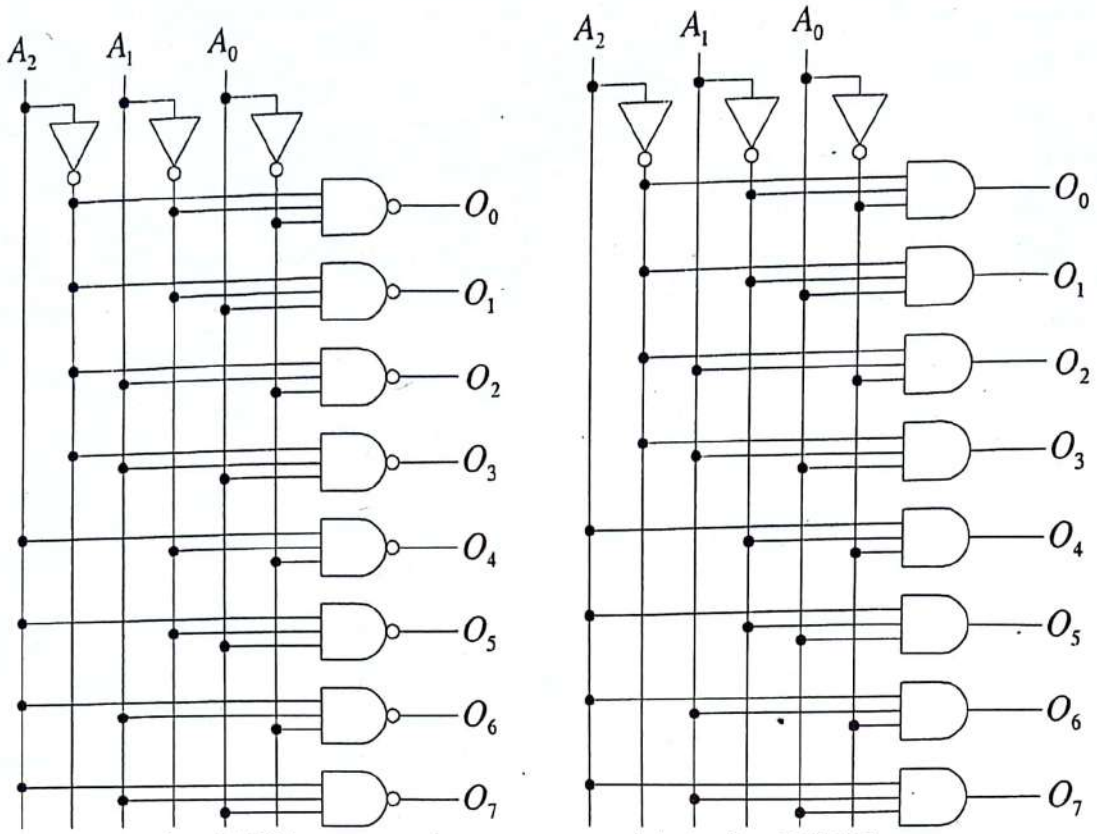



**Figure 8.5** Logic implementation of binary-to-octal decoder (3-to-8-line decoder or 1-of-8 decoder).

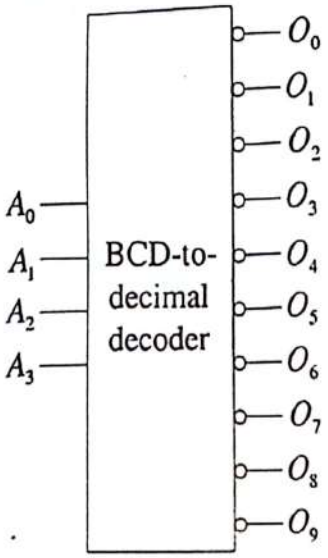
The decoders with binary inputs are called $n$-to-$m$ line decoder (or 1-of-$m$ decoders) where $m \leq 2^n$. Their purpose is to generate the $2^n$ (or less) minterms (active-HIGH output) or maxterms (active-LOW output) of $n$ input variables.

## BCD-to-decimal decoder

Sometimes decoders are designed to accept coded binary number such as BCD number or Excess-3 number. The design of a BCD-to-decimal decoder with active-LOW output is given here. The block diagram and truth table of a BCD-to-decimal decoder with active-LOW output are given in Figure 8.6. Note that for the invalid BCD inputs (1010 to 1111) no output line is activated. The Boolean functions for the outputs can be written from the truth table as

$$O_0 = A_3 + A_2 + A_1 + A_0 = (A_3' A_2' A_1' A_0')'$$
$$O_1 = A_3 + A_2 + A_1 + A_0' = (A_3' A_2' A_1' A_0)'$$
$$O_2 = A_3 + A_2 + A_1' + A_0 = (A_3' A_2' A_1 A_0')'$$
$$O_3 = A_3 + A_2 + A_1' + A_0' = (A_3' A_2' A_1 A_0)'$$
$$O_4 = A_3 + A_2' + A_1 + A_0 = (A_3' A_2 A_1' A_0')'$$
$$O_5 = A_3 + A_2' + A_1 + A_0' = (A_3' A_2 A_1' A_0)'$$

$$O_6 = A_3 + A_2' + A_1' + A_0 = (A_3' A_2 A_1 A_0')'$$
$$O_7 = A_3 + A_2' + A_1' + A_0' = (A_3' A_2 A_1 A_0)'$$
$$O_8 = A_3' + A_2 + A_1 + A_0 = (A_3 A_2' A_1' A_0')'$$
$$O_9 = A_3' + A_2 + A_1 + A_0' = (A_3 A_2' A_1' A_0)'$$




(a) Block diagram

| Inputs | | | | Outputs | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $A_3$ | $A_2$ | $A_1$ | $A_0$ | $O_9$ | $O_8$ | $O_7$ | $O_6$ | $O_5$ | $O_4$ | $O_3$ | $O_2$ | $O_1$ | $O_0$ |
| 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 |
| 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 1 |
| 0 | 0 | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 1 |
| 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 1 | 1 |
| 0 | 1 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 1 |
| 0 | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 1 |
| 0 | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 1 |
| 0 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 1 | 0 | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 1 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 1 | 1 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 1 | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 1 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |

(b) Truth table

**Figure 8.6** Block diagram and truth table of BCD-to-decimal decoder with active-LOW output.

The logic implementation of the BCD-to-decimal decoder with active-LOW output is shown in Figure 8.7.

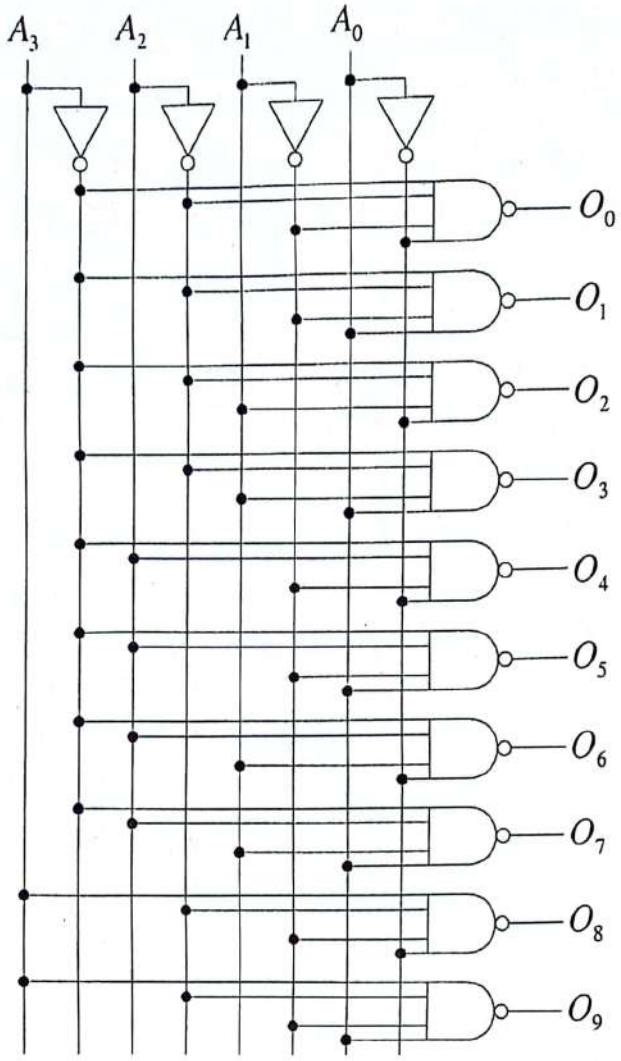


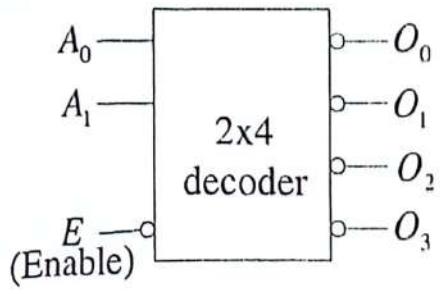

**Figure 8.7** Logic implementation of a BCD-to-decimal decoder with active-LOW output.

### 8.3.2 Decoders with Enable Input

Some decoders have enable input that is used to control the operation of the decoder. If the enable input is active, the output corresponding to the input is activated. If the enable input is not active, no output is activated. The block diagram and truth table of a 2-to-4-line decoder with active-LOW output and active-LOW enable is shown in Figure 8.8. From truth table the Boolean functions for the outputs can be written as

$$O_0 = E + A_1 + A_0 = (E'A_1'A_0')'$$
$$O_1 = E + A_1 + A_0' = (E'A_1'A_0)'$$
$$O_2 = E + A_1' + A_0 = (E'A_1A_0')'$$
$$O_3 = E + A_1' + A_0' = (E'A_1A_0)'$$

The logic implementation of the 2-to-4-line decoder with active-LOW output and active-LOW enable input is shown in Figure 8.9.




(a) Block diagram

| Inputs | | | Outputs | | | |
|---|---|---|---|---|---|---|
| $E$ | $A_1$ | $A_0$ | $O_3$ | $O_2$ | $O_1$ | $O_0$ |
| 0 | 0 | 0 | 1 | 1 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 1 |
| 0 | 1 | 0 | 1 | 0 | 1 | 1 |
| 0 | 1 | 1 | 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 1 | 1 | 1 | 1 |
| 1 | 0 | 1 | 1 | 1 | 1 | 1 |
| 1 | 1 | 0 | 1 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 | 1 | 1 | 1 |

(b) Truth table

**Figure 8.8** Block diagram and truth table of 2-to-4-line decoder with active-LOW output and active-LOW enable input.

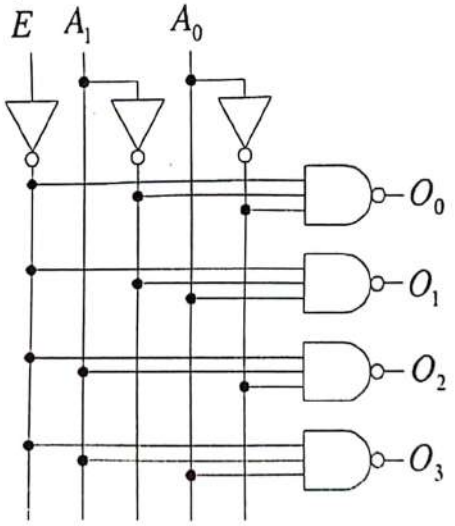


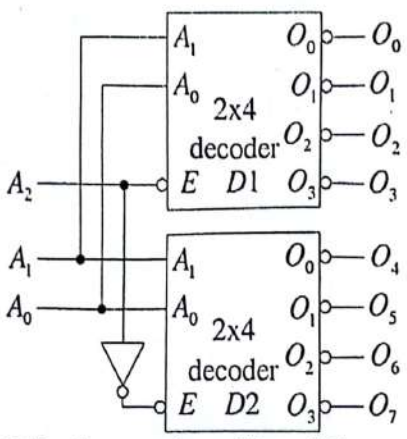

**Figure 8.9** Logic implementation of 2-to-4-line decoder with active-LOW output and active-LOW enable input.

### 8.3.3 Expansion of Decoder

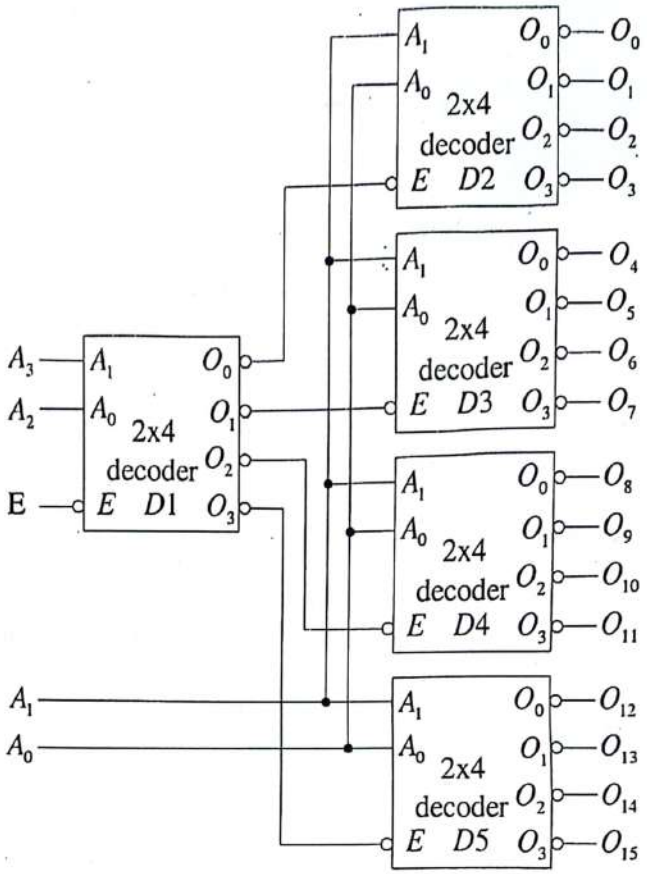
To meet the larger inputs needs, decoders can be expanded by using *enable* input line. Figure 8.10(a) shows a 3-to-8-line decoder and Figure 8.10(b) shows a 4-to-16-line decoder using 2-to-4-line decoders.

In Figure 8.10(a), when $A_2 = 0$, decoder $D1$ is enabled and decoder $D2$ is disabled. The output corresponding to $A_1A_0$ input of the decoder $D1$ is activated. When $A_2 = 1$, decoder $D2$ is enabled and decoder $D1$ is disabled. The output corresponding to $A_1A_0$ input of the decoder $D2$ is activated. For example, if $A_2A_1A_0 = 101$, decoder $D2$ will be enabled since $A_2 = 1$ and the output $O_5$ of

the decoder $D2$ will be activated since $A_1A_0 = 01$. Therefore, output $O_5$ of the combined decoder will be activated.




(a) Implementation of 3-to-8-line decoder with 2-to-4-line decoders




(b) Implementation of 4-to-16-line decoder with 2-to-4-line decoders

**Figure 8.10** Expansion of decoder.

In Figure 8.10(b), when high-order bits $A_3A_2 = 00$, the output $O_0$ of the decoder $D1$ will be LOW and the decoder $D2$ will be enabled. The output of the decoder $D2$ corresponding to the low-order bits $A_1A_0$ will be activated. Similarly, when $A_3A_2 = 01$, the decoder $D3$ will be enabled and the output of the decoder $D3$ corresponding to $A_1A_0$ will be activated. When $A_3A_2 = 10$, decoder $D4$ will be enabled and the output of the decoder $D4$ corresponding to $A_1A_0$ will be activated. Finally, when $A_3A_2 = 11$, decoder $D5$ will be enabled and the output of the decoder $D5$ corresponding to $A_1A_0$ will be activated. For example, when $A_3A_2A_1A_0 = 0110$, that is high-order bits $A_3A_2 = 01$ and low-order bits $A_1A_0 = 10$, the $O_1$ output of the decoder $D1$ will be LOW, which will enable the decoder $D3$. Then the output $O_2$ of the decoder $D3$ will be activated, which is the output $O_6$ of the combined decoder.

### 8.3.4 Combinational Logic Implementation with Decoder

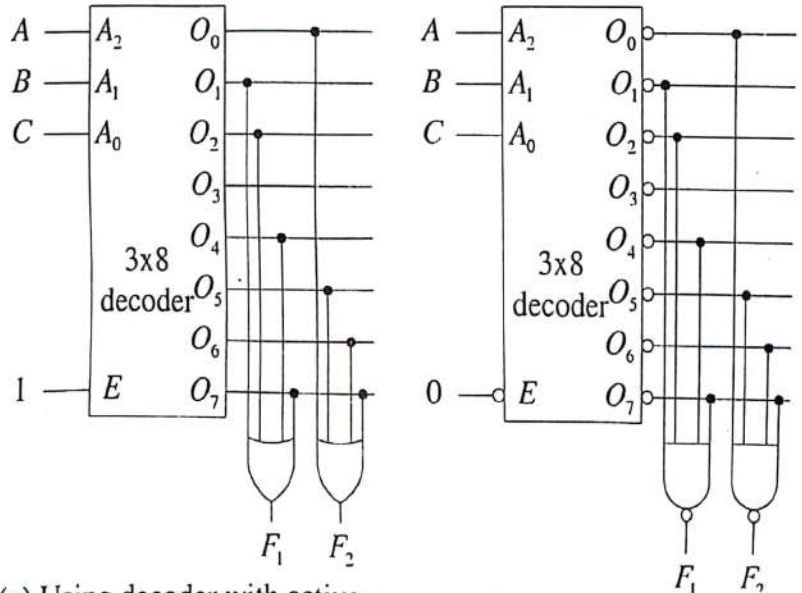
A decoder provides the $2^n$ minterms (active-HIGH output) or maxterms (active-LOW output) of $n$ input variables. Since, any Boolean function can be expressed as sum of minterms (canonical sum of products) or product of maxterms (canonical product of sums), decoders can be used to implement Boolean function in conjunction with some external gates. In this implementation method, any combinational circuit with $n$ inputs and $m$ outputs can be implemented with an $n$- to-$2^n$-line decoder and $m$ OR/NAND or AND/NOR gates.

#### Implementation of Canonical Sum of Products Function

For implementing an $n$-variable canonical sum of products function, the $n$ inputs of the function are applied to the $n$ inputs of an $n$- to-$2^n$-line decoder with active-HIGH outputs. As the outputs of the decoder generate the minterms of an $n$-variable function, the outputs of the decoder corresponding to the minterms of the function are ORed with an OR gate to form the sum. Implementation of the following 3-variable functions using a 3-to-8-line decoder with active-HIGH output is shown in Figure 8.11(a).

$$F_1(A,B,C) = \sum(1,2,4,7)$$
$$F_2(A,B,C) = \sum(0,5,6,7)$$

The canonical sum of products function can also be implemented using decoder with active-LOW output. In this decoder, the outputs represent the complement of the minterms. In this implementation, the complemented minterms are combined with a NAND gate to form the sum of minterms. The procedure is obvious from DeMorgan's theorem. Implementation of the above functions using a 3-to-8-line decoder with active-LOW output is shown in Figure 8.11(b).




(a) Using decoder with active-HIGH outpt

(a) Using decoder with active-LOW outpt

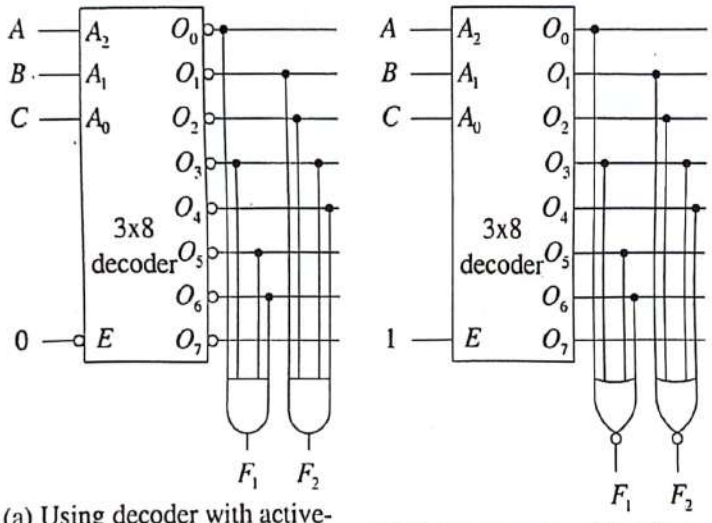
**Figure 8.11** Implementation of canonical sum of product functions $F_1(A,B,C) = \sum(1,2,4,7)$ and $F_2(A,B,C) = \sum(0,5,6,7)$ using decoders.

## Implementation of Canonical Product of Sums Function

For implementing an $n$-variable canonical product of sums function, the $n$ inputs of the function are applied to the $n$ inputs of an $n$-to-$2^n$-line decoder with active-LOW outputs. As the outputs of the decoder generate the maxterms of an $n$-variable function, the outputs of the decoder corresponding to the maxterms of the function are ANDed with an AND gate to form the product. Implementation of the following 3-variable functions using a 3-to-8-line decoder with active-LOW output is shown in Figure 8.12(a).

$$F_1(A,B,C) = \prod(0,3,5,6)$$
$$F_2(A,B,C) = \prod(1,2,3,4)$$

The canonical product of sums function can also be implemented using decoders with active-HIGH outputs. In this decoder, the outputs represent the complement of the maxterms. In this implementation, the complemented maxterms are combined with a NOR gate to form the product of maxterms. The procedure is obvious from DeMorgan's theorem. Implementation of the above functions using a 3-to-8-line decoder with active-HIGH output is shown in Figure 8.12(b).




(a) Using decoder with active-LOW outpt

(a) Using decoder with active-HIGH outpt

**Figure 10.12** Implementation of canonical product of sums function $F_1(A,B,C) = \prod(0,3,5,6)$ and $F_2(A,B,C) = \prod(1,2,3,4)$ using decoders.

If the number of minterms in a function is less than $2^n/2$, then the implementation of Figure 8.11 is suitable, otherwise the implementation of Figure 8.12 is suitable.

The decoder method can be used to implement any combinational circuit. However, its implementation must be compared with all other possible implementations to determine the best solution. In some cases this method may provide the best implementation, especially if the combinational circuit has many outputs and if each output function is expressed with a small number of minterms or maxterms.

### 8.3.5 Available MSI Decoder ICs

List of available MSI decoder ICs are given in Table 8.2.

**Table 8.2** Available MSI decoder ICs.

| IC No. | Description |
|---|---|
| 74139 | Dual 2-to-4-line decoder with active-LOW output |
| 74155 | Dual 2-to-4-line decoder with active-LOW output |
| 74156 | Dual 2-to-4-line decoder with active-LOW output (open-collector) |
| 74138 | 3-to-8-line decoder with active-LOW output |
| 74154 | 4-to-16-line decoder with active-HIGH output |
| 7442 | BCD-to-decimal decoder with active-LOW output. |

IC 74138 is introduced here. The pin assignment of IC 74138 is shown in Figure 8.13. $A_0$ through $A_2$ are three address lines and $O_0'$ through $O_7'$ are eight active-LOW outputs. It has two active-LOW ( $E_1'$ and $E_2'$) and one active-HIGH ( $E_3$ ) enable inputs internally ANDed to produce a single enable signal.

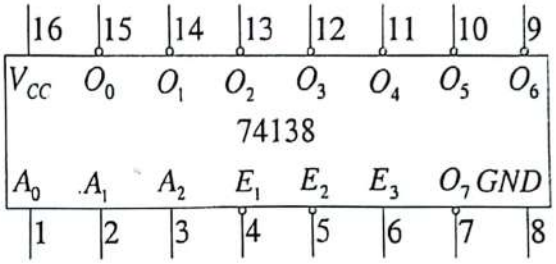



**Figure 8.13** IC type 74138 3-to-8-line decoder with active-LOW output.

## 8.4 Code Converters

There are a wide variety of binary codes used in digital systems. Some of these codes are BCD, Excess-3, etc. Different digital systems use different codes. It is sometimes necessary to use the output of one system as the input to another. Therefore, it is required to convert from one code to another. This sort of code converters are combinational circuits and may be regarded as special type of encoders, which encodes from one form to another. Code converters can be designed using classical method of combinational logic design or using MSI circuits. Design of BCD-to-Excess-3 code converter is given here.

The block diagram and truth table for BCD-to-Excess-3 code converter is shown in Figure 8.14. Note that the input combinations 1010 to 1111 are invalid BCD code and will not appear in the input. Therefore, their outputs are don't care states. The output functions are simplified using K-maps in Figure 8.15, which gives

$$E_0 = B_0'$$

$$E_1 = B_1'B_0' + B_1B_0$$

$$E_2 = B_2'B_0 + B_2'B_1 + B_2B_1'B_0'$$

$$E_3 = B_3 + B_2B_0 + B_2B_1$$